

Role-Based Authorization & Security Infrastructure

1. Feature Name

Role-Based Authorization & Security Infrastructure

2. Purpose

- **Problem Solved:** A multi-role learning platform (Admin, Instructor, Student) must restrict which actions each user may perform and protect the API from unauthorized access.
 - **Why It Exists:** Every controller endpoint needs a consistent, maintainable way to say "only admins", "admin or instructor", or "the authenticated user". This infrastructure centralizes identity registration, JWT bearer authentication, role claims, authorization policies, CORS, and secret configuration.
 - **Role in Application:** It is the security backbone. Authentication (Feature 1) establishes *who* the user is; this feature establishes *what* they are allowed to do. All controllers rely on it via `[Authorize]` attributes and named policies.
-

3. What the User Can Do

This is infrastructure; users do not interact with it directly. It manifests as:

- **All roles:** Only reach authenticated areas; route guards in the frontend (ProtectedRoute, AdminRoute, InstructorRoute, GuestRoute) control page visibility.
 - **Admins:** Can access Admin-only endpoints (user management, question banks, moderation, audit browsing).
 - **Instructors & Admins:** Can access instructor-level endpoints (create learning paths, quizzes, classrooms, assignments).
 - **Students:** Can access student endpoints (enroll, take quizzes, submit assignments, community).
-

4. Feature Workflow

[Frontend route navigation]

- Route guard checks Redux auth state + user roles
- Unauthorized → redirected to /login (ProtectedRoute) or "not found" (role guards)

[Frontend API request]

- apiClient attaches "Bearer <accessToken>" header
- Backend JwtBearer middleware validates the token (issuer, audience, lifetime, signature)
- Endpoint's [Authorize] / policy runs against the token's role claims
- Allowed → controller executes → service performs additional ownership checks
- Denied → 401 (unauthenticated) or 403 (forbidden)

5. How It Works Internally

Identity Setup

Configured in `backend/Program.cs`:

- `AddIdentity<User, IdentityRole>` with EF Core stores and default token providers.
- Password policy: digit, lowercase, uppercase required; length ≥ 8 ; no non-alphanumeric required; unique email.
- Lockout: 5 failed attempts → 5-minute lockout.

JWT Bearer Authentication

`AddAuthentication(...).AddJwtBearer(...)`:

- Default authenticate/challenge scheme is JWT Bearer.
- `TokenValidationParameters`: `ValidateIssuer`, `ValidateAudience`, `ValidateLifetime`, `ValidateIssuerSigningKey` all true; valid issuer/audience from `JwtSettings`; signing key = symmetric key from the JWT secret; `ClockSkew` = `TimeSpan.Zero`.
- Access tokens carry role claims (`ClaimTypes.Role`) generated by `JwtTokenGenerator.GenerateAccessToken` (one claim per role).

Authorization Policies

Two named policies are registered:

```
.AddPolicy("AdminOnly", p => p.RequireRole("Admin"))
.AddPolicy("InstructorOrAdmin", p => p.RequireRole("Admin", "Instructor"))
```

Controllers combine these with `[Authorize(Policy = "...")]` and `[Authorize(Roles = "...")]`.
Examples:

- `AdminController` and question-bank management use `[Authorize(Policy = "AdminOnly")]`.
- Quiz/path/classroom authoring uses `[Authorize(Policy = "InstructorOrAdmin")]`.
- Student actions use plain `[Authorize]`.

Role Seeding

`RoleSeeder.SeedAsync` (`backend/Data/Seeders/RoleSeeder.cs`) creates the roles `Admin`, `Instructor`, `Student` if they don't exist. `AdminSeeder` creates/updates a Super Admin user (`admin@learnpath.com`, password `Admin@123`, `IsSuperAdmin = true`, assigned the `Admin` role) during startup seeding.

Service-Level Re-checks (defense in depth)

Beyond endpoint attributes, services re-verify ownership using the user id from the JWT claim `ClaimTypes.NameIdentifier`. Examples seen across the codebase:

- Instructors can only manage learning paths/classes/quizzes they created (`CreatedById == userId`).
- Students can only access their own attempts, progress, and submissions (`UserId == userId`).
- `IsSuperAdmin` gates role changes and global audit-history browsing in `AdminService`.

Frontend Route Guards

In `frontend/src/routes/`:

- `ProtectedRoute.tsx` — redirects unauthenticated users to `/login` (preserving the intended destination via `state.from`).
- `AdminRoute.tsx`, `InstructorRoute.tsx` — redirect non-matching-role users away.
- `GuestRoute.tsx` — redirects already-authenticated users away from login/register.
- Role values come from `frontend/src/constants/roles.ts` (`Admin`, `Instructor`, `Student`).

CORS

`LearnPathCors` policy allows the origins listed in `AllowedOrigins` (from `appsettings/.env`), any header, any method, and credentials.

Secret Configuration

`.env` values are loaded at startup (`DotNetEnv`), mapped into configuration keys (`JwtSettings:Secret`, `AiOptions:*`, `ConnectionStrings:DefaultConnection`), and override

appsettings.json . Startup fails fast with a clear message if required variables (LEARNPATH_JWT_SECRET , LEARNPATH_DB_CONNECTION , LEARNPATH_NVIDIA_API_KEY) are missing.

6. Frontend Implementation

Layer	File Path	Responsibility
Guard	frontend/src/routes/ProtectedRoute.tsx	Redirects unauthenticated users to login
Guard	frontend/src/routes/AdminRoute.tsx	Restricts a subtree to Admin role
Guard	frontend/src/routes/InstructorRoute.tsx	Restricts a subtree to Instructor/Admin roles
Guard	frontend/src/routes/GuestRoute.tsx	Keeps logged-in users off auth pages
Config	frontend/src/constants/roles.ts	Role name constants
Hooks	frontend/src/hooks/useAuth.ts	isAdmin , isInstructor , isStudent flags
Service	frontend/src/services/apiClient.ts	Attaches Bearer token to every request

7. Backend Implementation

Layer	File Path	Responsibility
Config	backend/Program.cs	Identity, JWT bearer, policies, CORS, secrets, seeding
Seed	backend/Data/Seeders/RoleSeeder.cs	Creates Admin/Instructor/Student roles + Super Admin
JWT	backend/Authentication/Jwt/JwtTokenGenerator.cs	Adds role claims to access tokens

Layer	File Path	Responsibility
JWT	backend/Authentication/Jwt/JwtSettings.cs	Token issuer/audience/secret config
Entity	backend/Entities/User.cs	IsSuperAdmin, Status, identity inheritance

8. Database Implementation

Entity/Table	Purpose	Important Relationship
AspNetRoles	Role rows (Admin, Instructor, Student)	PK role id
AspNetUserRoles	User ↔ Role assignment	FK user id, FK role id
AspNetUsers	Users incl. IsSuperAdmin, Status	FK from AspNetUserRoles
AspNetUserClaims	(Identity) user claims	FK user id

- Roles are created idempotently at startup by RoleSeeder.
- AspNetUserRoles rows are added on registration (Student) and when admins change roles.

9. Security & Authorization

- JWT bearer authentication with issuer/audience/lifetime/signature validation and zero clock skew.
- Named policies (AdminOnly, InstructorOrAdmin) plus [Authorize(Roles = ...)] for role-level checks.
- Service-level ownership checks using the sub claim (ClaimTypes.NameIdentifier) — never a client-supplied user id.
- IsSuperAdmin flag gates the most privileged actions (role changes, global audit history).
- CORS restricted to configured origins with AllowCredentials.
- Secrets live in .env (git-ignored); startup fails if required secrets are missing.
- FluentValidation + Identity policies validate inputs centrally.

10. Important Business Rules

1. **Default role:** Self-registration grants `Student`.
 2. **Role hierarchy by policy:** Admin-only actions require the `Admin` role; authoring actions require `Admin` or `Instructor`; basic actions require any authenticated user.
 3. **Super Admin:** Only `IsSuperAdmin == true` users may change roles and browse all users' audit history.
 4. **Ownership:** Instructors are bound to resources they created; students to their own records.
 5. **Lockout:** 5 failed password attempts lock the account for 5 minutes.
-

11. Example of Internal Execution

Step 1: An admin opens the user-management page

- **Frontend:** `AdminRoute` verifies the Redux user has the `Admin` role; otherwise the page is not reachable.
- **API:** `ApiClient` sends `Authorization: Bearer <token>` to `GET /api/v1/admin/users`.
- **Backend:** `JwtBearer` middleware validates the token; the `[Authorize(Policy = "AdminOnly")]` attribute checks the token's `role` claims contain `Admin`.
- **Service:** `AdminService` additionally requires `user.IsSuperAdmin` for role-changing operations.
- **Result:** The request proceeds; a non-admin user receives `403 Forbidden`.

Step 2: An instructor tries to update another instructor's learning path

- Endpoint allows `InstructorOrAdmin` (passes role check).
 - `LearningPathService` compares `path.CreatedById` with the `sub` claim of the token and rejects the operation with a "not owner" error.
-

12. Files Involved

Frontend

- `frontend/src/routes/ProtectedRoute.tsx`
- `frontend/src/routes/AdminRoute.tsx`
- `frontend/src/routes/InstructorRoute.tsx`
- `frontend/src/routes/GuestRoute.tsx`
- `frontend/src/constants/roles.ts`

- `frontend/src/hooks/useAuth.ts`
- `frontend/src/services/apiClient.ts`

Backend

- `backend/Program.cs`
- `backend/Data/Seeders/RoleSeeder.cs`
- `backend/Authentication/Jwt/JwtTokenGenerator.cs`
- `backend/Authentication/Jwt/JwtSettings.cs`
- `backend/Entities/User.cs`
- `backend/Entities/UserStatus.cs`
- `backend/appsettings.json`
- `backend/.env.example`

13. Functionality

- Role seeding (Admin, Instructor, Student) + Super Admin provisioning
- JWT bearer authentication with strict token validation
- Role-claim-based authorization (AdminOnly, InstructorOrAdmin policies)
- `[Authorize(Roles)]` attribute checks across controllers
- Service-level ownership re-checks
- Frontend route guards per role
- CORS with credentials
- `.env`-based secret configuration with startup validation
- Account lockout and password policy via Identity